ZTF-Orchestrator

User Guide

Version 1.2.8

Document Title	Initial Configuration & Operation
Version	1.2.8
Date	2026-06-02
Author	John Goulden
Classification	Public

Table of Contents

1. Overview

ZTF-Orchestrator is a web-based interface for the Nutanix ZeroTouch Framework. It provides a visual operator interface for automating Nutanix infrastructure deployments including cluster creation, node imaging, Prism Central deployment, and workload provisioning without requiring command-line access to the underlying Python framework.

Key capabilities:

- 14 pre-built workflow templates with guided form-based configuration
- 61 individual atomic scripts organised across 12 categories
- Global credential and IPAM configuration (Local / CyberArk / Infoblox)
- Config file management with automatic 5-version backup and restore
- Durable execution jobs with live terminal output, persisted logs, cancellation, and restart recovery
- PostgreSQL-backed state by default for Docker deployments, with file-backed mode still available for local testing

Deployment and Appliance Model

ZTF-Orchestrator can be run directly from source, with Docker Compose, or as a Linux VM appliance. For shared team or AHV deployments, the recommended model is a small Linux VM running Docker Compose, PostgreSQL, and the published ZTF-Orchestrator container image.
The repository includes an appliance kit with first-boot scripts, systemd units, cloud-init examples, and a reference Packer template for building an AHV-importable QCOW2. Packer is optional and is only required if you want to build and publish a prebuilt appliance image. It is not required for normal Docker deployment.
Large VM images should be published as GitHub Release artifacts, not committed to the repository. Secrets such as the PostgreSQL password, application admin password, and Nutanix credentials are generated or configured on the deployed appliance and must not be baked into Git.

- Execution history with re-run capability
- Workflow dry-run for pre-flight validation before committing
- Role-based access control: admin, operator, viewer
- Scheduled workflow execution with cron-based run times
- Parallel multi-site execution for up to 10 site-specific configs
- Approval gates for controlled operations
- Named connection profiles for Prism Central, Foundation Central, Prism Element, NCM, directory, IPAM, DNS, and NTP defaults
- Configuration drift detection against last applied or pasted current state
- Webhook notifications for workflows, scripts, schedules, pipelines, approvals, and parallel runs

2. Signing In

Open a browser and navigate to http://localhost:5001 (or the configured port). On first launch the server generates a random admin password and prints it to the terminal.

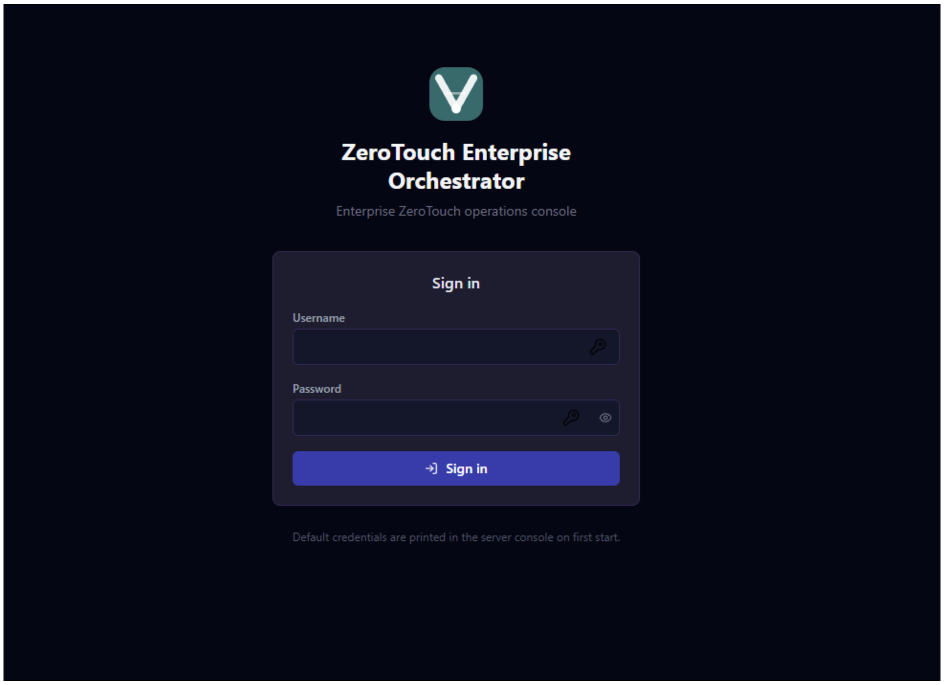

Figure: Login screen

Enter the username admin and the password printed in the server console, then click Sign in.

> **First Run:** Copy the password from the terminal before navigating to the browser. If missed, delete users.json from ~/.ztf-ui/ and restart the server to regenerate credentials.

3. Dashboard

The Dashboard is the home screen. It shows system health, execution statistics, common workflow shortcuts, recent execution history, and first-run onboarding actions when no executions have been recorded yet.

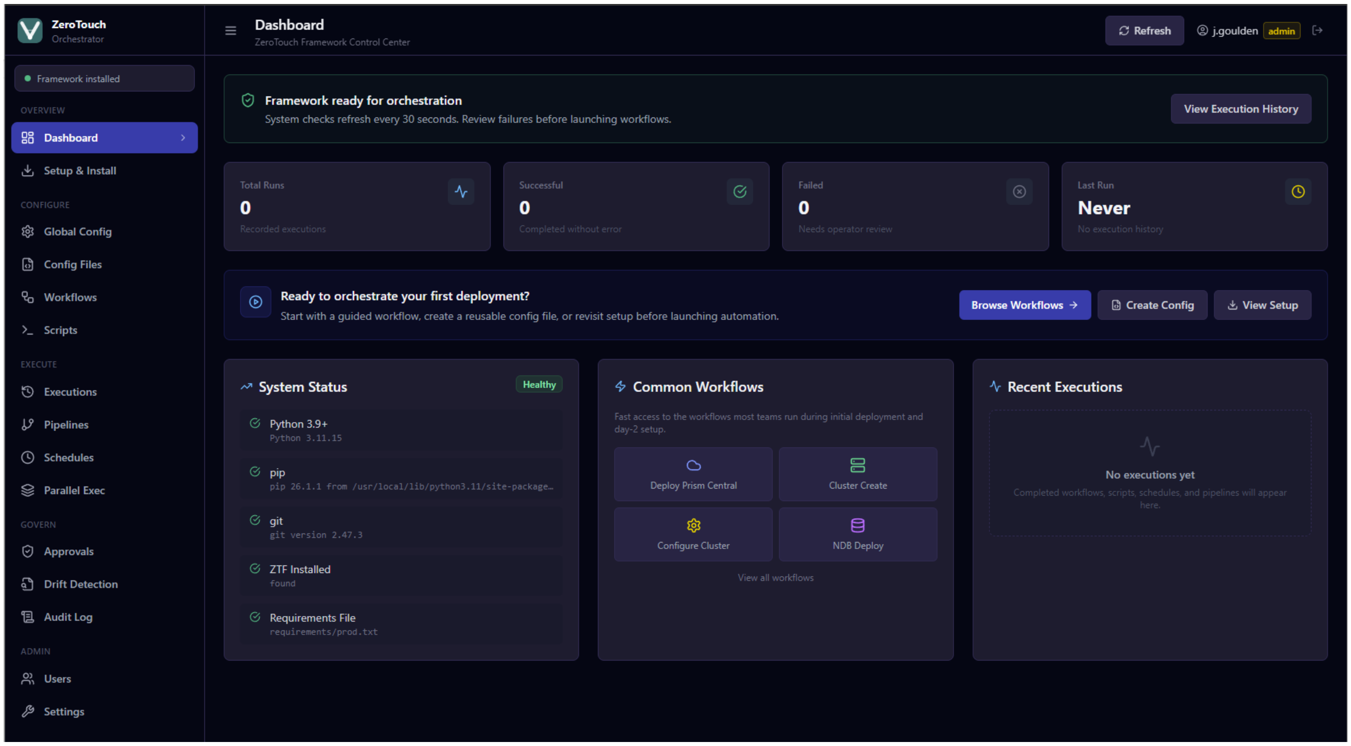

Figure: Dashboard grouped sidebar, health banner, execution stats, first-run CTA, common workflows, and recent executions

System Status

The System Status panel confirms that all prerequisites are met:

- Python 3.9+: the Python interpreter on PATH
- Pip: package manager
- Git: required for the Setup & Install wizard
- ZTF Installed: confirms main.py is found at the configured ZTF path
- Requirements File: confirms the ZTF dependency file exists
- State Backend: confirms whether the appliance is using PostgreSQL or file-backed state and shows the database location when configured

> **Note: The green Framework installed badge in the top-left sidebar indicates ZTF is ready to use. The Dashboard System Status panel also shows the active state backend so operators can immediately confirm whether the console is running against PostgreSQL or file-backed storage.**

Auto-refresh

The Total Runs, Successful, and Failed metric tiles are clickable when their count is greater than zero. Each opens Execution History with the relevant filter applied so operators can move directly from the dashboard summary to the underlying records.

First-run onboarding

When there is no execution history, the Dashboard displays a full-width onboarding banner: Ready to orchestrate your first deployment? The banner routes directly to Browse Workflows, Create Config, and View Setup so new users have an obvious next step.

Navigation layout

The sidebar is grouped by operational intent: Overview, Configure, Execute, Govern, and Admin. This keeps setup, configuration, execution, governance, and administration tasks visually separated as the product grows.

Common Workflows

The Common Workflows panel provides fast access to frequently used deployment and day-2 workflows: Deploy Prism Central, Cluster Create, Configure Cluster, and NDB Deploy. The full workflow library remains available from Workflows.

The dashboard refreshes automatically every 30 seconds. Use the Refresh button in the top-right corner to trigger an immediate update.

4. Setup & Install

Use the Setup & Install page to install or reinstall the ZeroTouch Framework. This page is only required once during initial deployment, or when updating the framework.

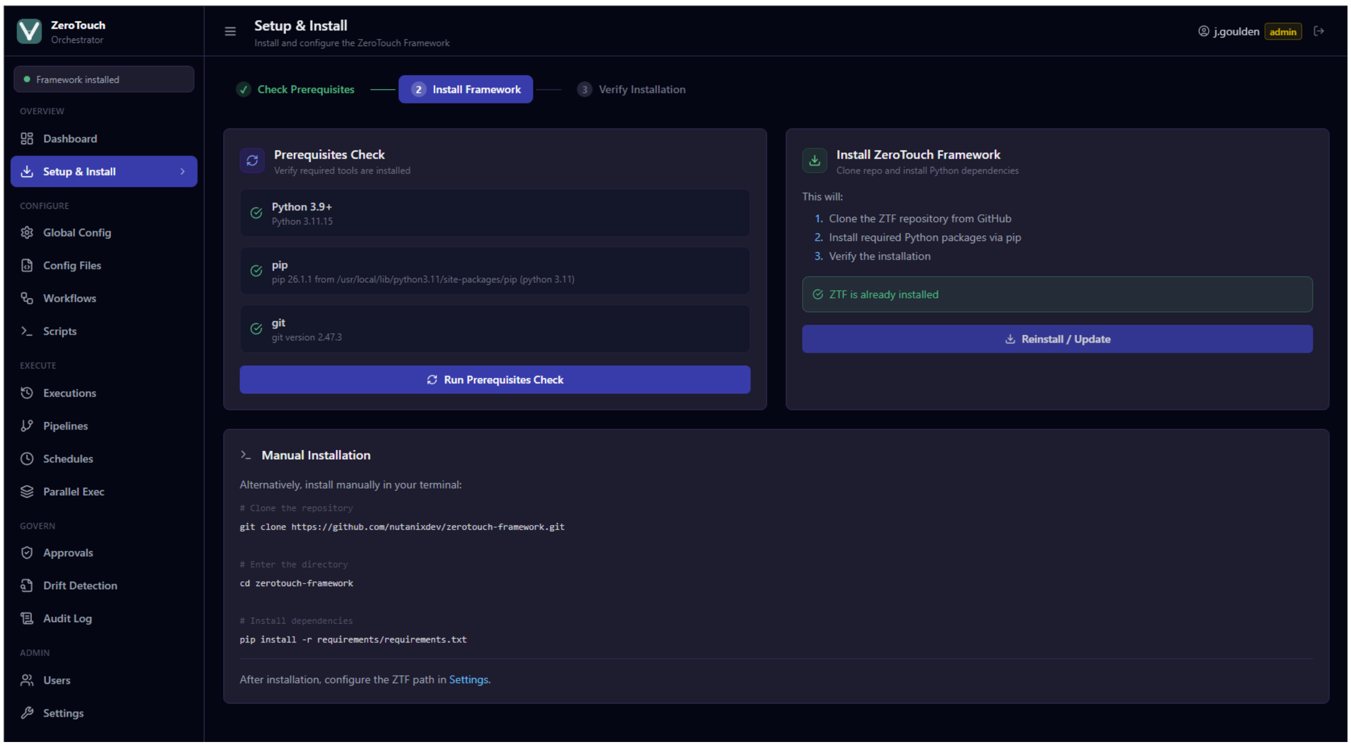

Figure: Setup & Install prerequisites check and installation

Prerequisites Check

Once the prerequisite check succeeds, the result remains green while navigating during the same browser session. This prevents the setup stepper from appearing incomplete when returning to Setup & Install later in the session.

Click Run Prerequisites Check to verify Python, pip, and git are available. Each item is confirmed with a green tick before installation proceeds.

Install Framework

Click Install Framework to clone the ZTF repository from GitHub and install its Python dependencies. Progress streams live in the terminal panel. For air-gapped environments, see the Field Installation Guide.

5. Global Configuration

Global Config defines the credentials, vault type, and IP allocation method used across all workflows. Changes are saved to global.yml in the ZTF framework directory.

Credentials Tab

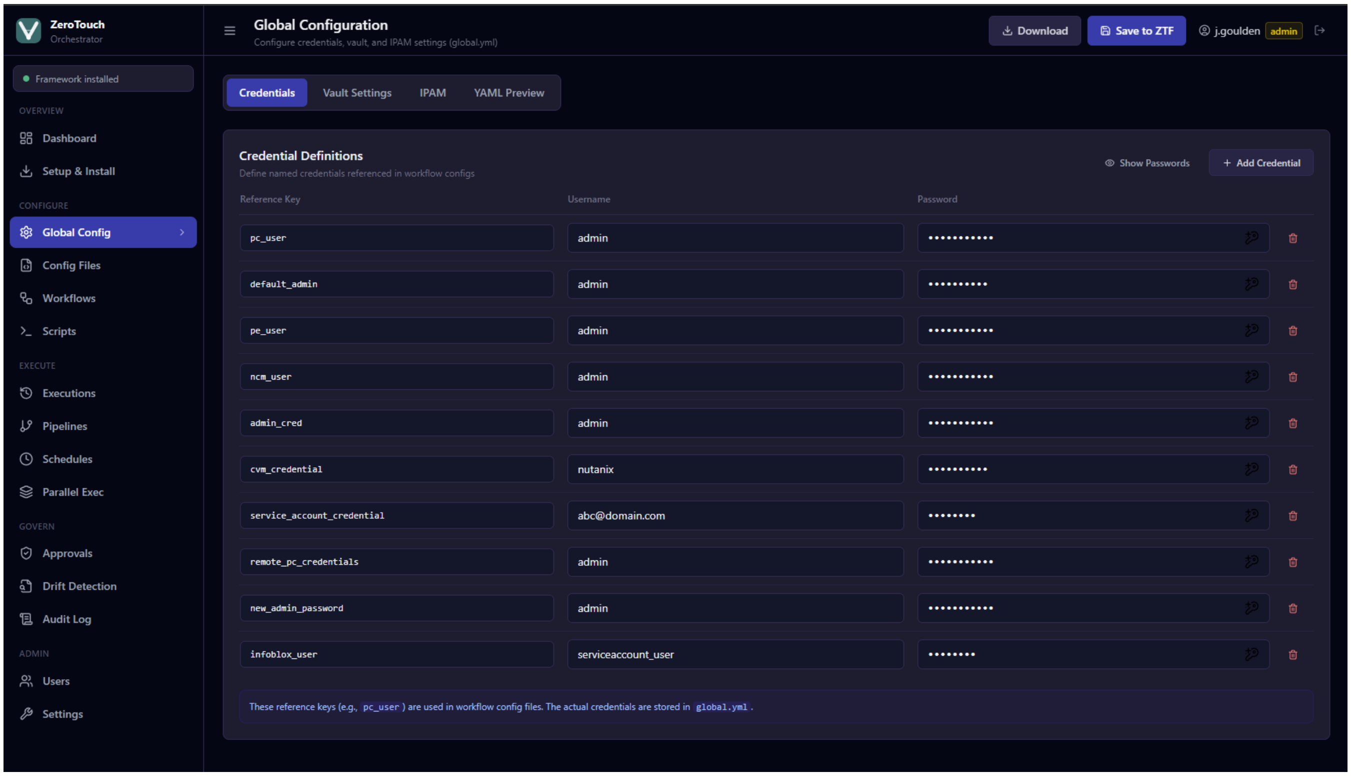

Figure: Global Config Credentials tab

Each row defines a named credential reference. The reference key (e.g. pc_user, cvm_credential) is used in workflow config files to look up the actual username and password stored here. Click Add Credential to add additional entries.

- Reference Key: the identifier used in workflow YAML files
- Username: the account username
- Password: stored securely in global.yml (masked by default)

Vault Settings Tab

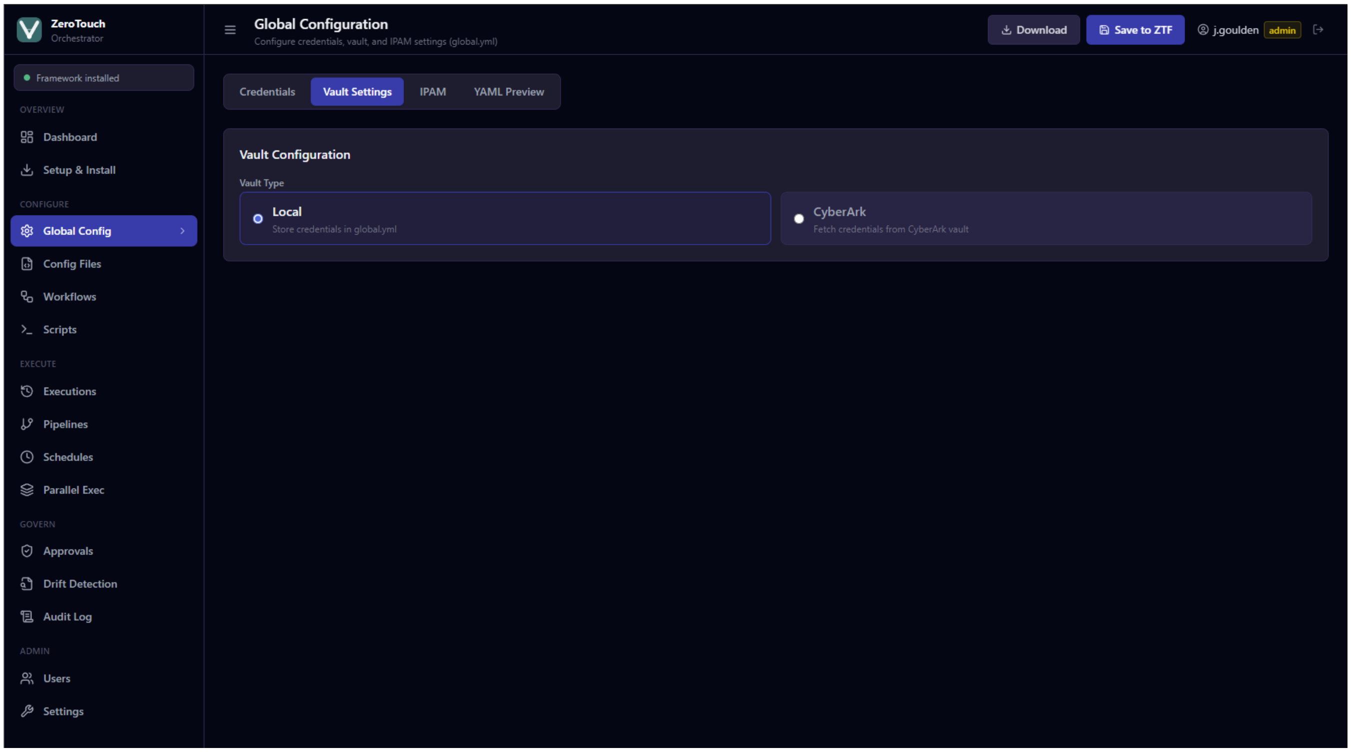

Figure: Global Config Vault Settings tab

Select the vault type for credential storage:

- Local: credentials stored directly in global.yml (default)
- CyberArk: credentials fetched at runtime from CyberArk; provide host, certificate, and key paths

> **Note:** The IPAM tab configures IP allocation: Static (manually specified per workflow) or Infoblox (automatic allocation). Click Save to ZTF to write global.yml to the framework directory.

6. Workflows

The Workflows page lists all 14 pre-built automation templates, grouped by category. Use the search bar or category filters to find a specific workflow.

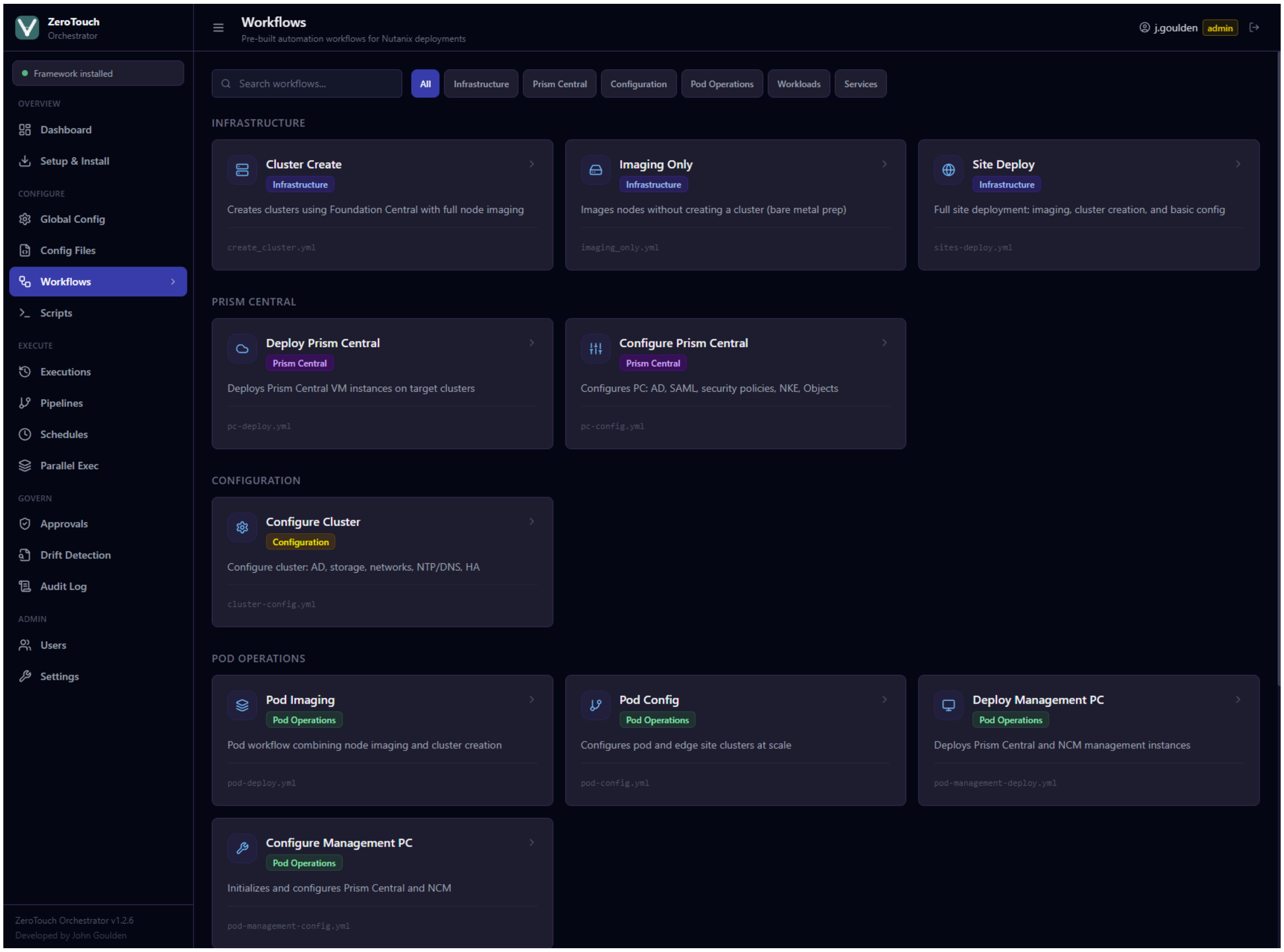

Figure: Workflows list all 14 templates grouped by category

Category	Workflows
Infrastructure	Cluster Create, Imaging Only, Site Deploy
Prism Central	Deploy Prism Central, Configure Prism Central
Configuration	Configure Cluster
Pod Operations	Pod Imaging, Pod Config, Deploy Management PC, Configure Management PC
Workloads	Calm VM Workloads, Edge AI Workload
Services	NDB Deploy
Lifecycle Management	LCM Update

7. Running a Workflow

Click any workflow card to open the workflow detail page.

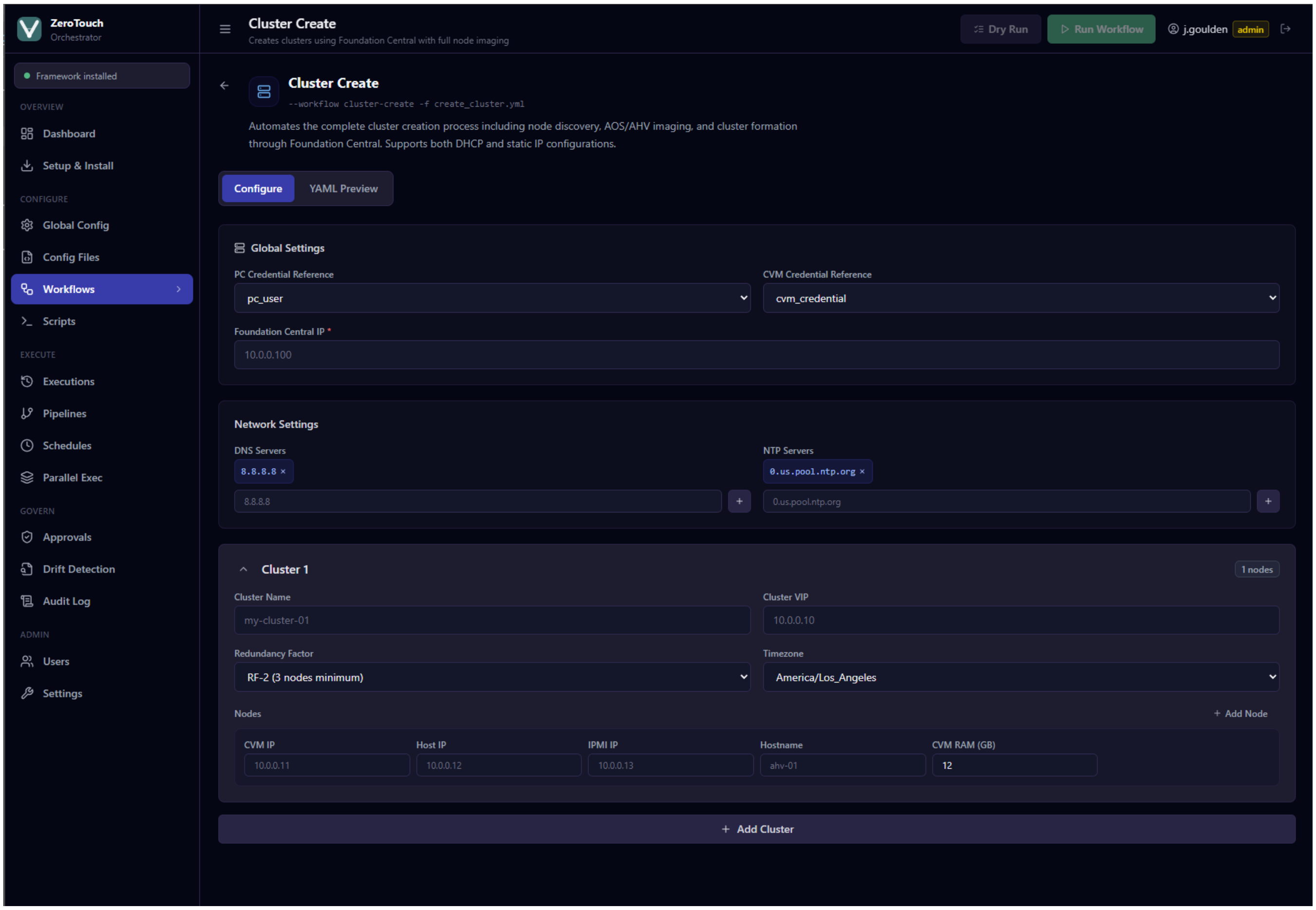

Figure: Workflow detail Cluster Create form with Dry Run and Run Workflow buttons

Configure Tab

Fill in the required fields in the Configure tab. Guided workflow forms now use the active Settings connection profile to pre-fill common values such as Prism Central, Foundation Central, Prism Element/CVM credentials, NCM endpoint, DNS, NTP, storage container, network name, image URLs, and hypervisor type where those fields apply.

YAML Preview Tab

Click YAML Preview to inspect the generated configuration before executing. Use Download Config to save the YAML file for offline use or manual execution.

Dry Run

Click Dry Run to run a pre-flight check without executing the workflow. The dry run validates:

- YAML structure: the config is valid and parseable
- Required fields: all mandatory workflow parameters are present
- IP address format: key IP fields contain valid addresses
- TCP reachability: key hosts (Foundation Central, Prism Central, etc.) are reachable on port 9440

Note: Dry Run fires pre-flight checks only. No changes are made to the Nutanix environment.

Run Workflow

Estimated Phase Progress

From v1.2.8, workflow and script executions show an estimated phase-based progress bar in the live execution modal and in Jobs / Queue. The percentage is an orchestration estimate based on queue state, configuration preparation, ZTF process launch, and observable ZeroTouch Framework output. It is not an authoritative Prism Central, Foundation Central, or Nutanix task percentage.
Future enhancement: if ZeroTouch Framework output exposes real Nutanix task IDs, ZTF-Orchestrator can enrich progress with task links or authoritative task status.

Click Run Workflow to execute. ZTF-Orchestrator creates a durable execution job, a background worker launches the ZeroTouch Framework subprocess, and the terminal modal streams the persisted job log. The workflow continues independently of the browser request; the terminal remains available to review output after completion.

8. Script Library

61 atomic scripts are available for targeted operations adding AD servers, creating subnets, deploying NKE clusters, and more. Scripts can be filtered by category or searched by name.

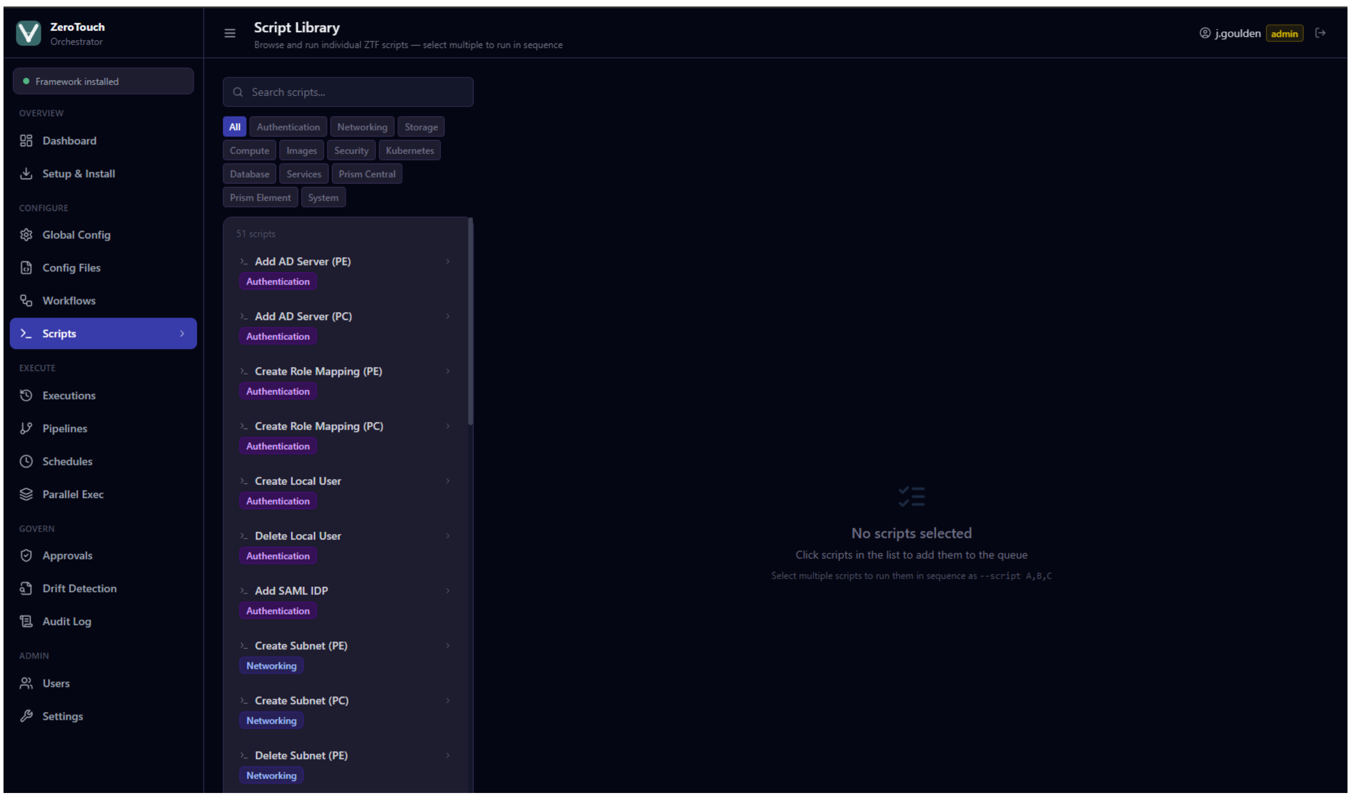

Figure: Script Library 61 atomic scripts across 12 categories

Select a script from the list, provide the required YAML configuration in the editor panel, then click Run Script. Output streams live in the terminal.

9. Config Files

The Config Files page manages YAML and JSON configuration files used by workflows and scripts. Files are stored in the configured config directory.

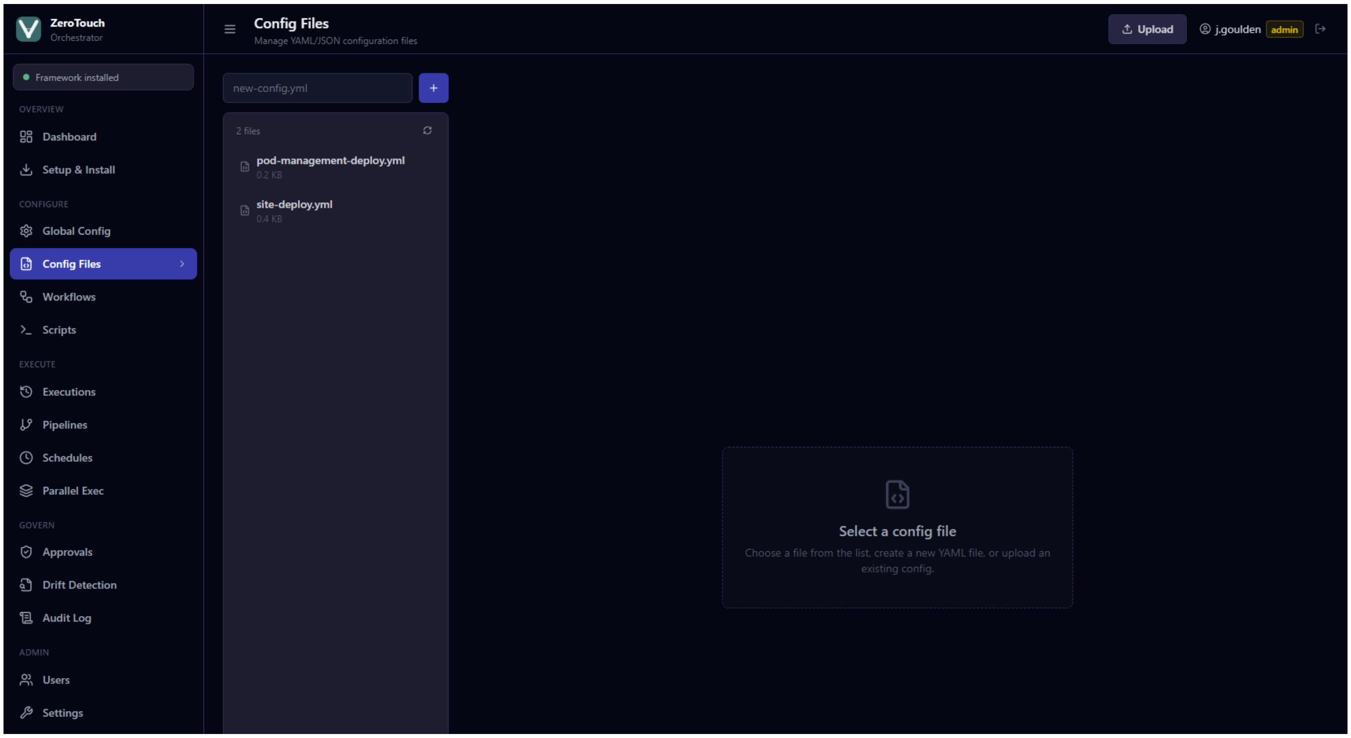

Figure: Config Files file list with editor and version history

Creating and Editing Files

- Type a filename in the input at the top-left and click + to create a new file
- Click any existing file to open it in the editor
- Edit the YAML/JSON content directly in the editor pane
- Click Save to write the file; a .bak.1 backup is created automatically
- Use Upload to import an existing config file from your local machine

Version History

Each file retains up to 5 previous versions automatically. If backups exist, a History button appears in the action bar showing version count. Click it to view all versions with timestamps and sizes.

- Click Restore on any version to revert the file to that state
- The current file is backed up before any restore operation

> **Note:** Config files are the input passed to workflows at execution time. A workflow reads its config file at the path shown in the YAML Preview tab.

10. Execution History

Jobs / Queue

Jobs / Queue shows durable workflow and script jobs, including queued, running, cancelling, success, failed, cancelled, and interrupted states. Each job retains persisted output, timestamps, return code, cancellation controls where permitted, and estimated phase progress introduced in v1.2.8.

Every workflow, script, schedule, pipeline, parallel run, and durable execution job is recorded in Execution History with status, timestamp, duration, user, and source type where applicable. Up to 1000 records are retained.

Execution jobs can be queued, running, successful, failed, cancelled, or interrupted. Interrupted indicates a service restart occurred while a job was queued or running.

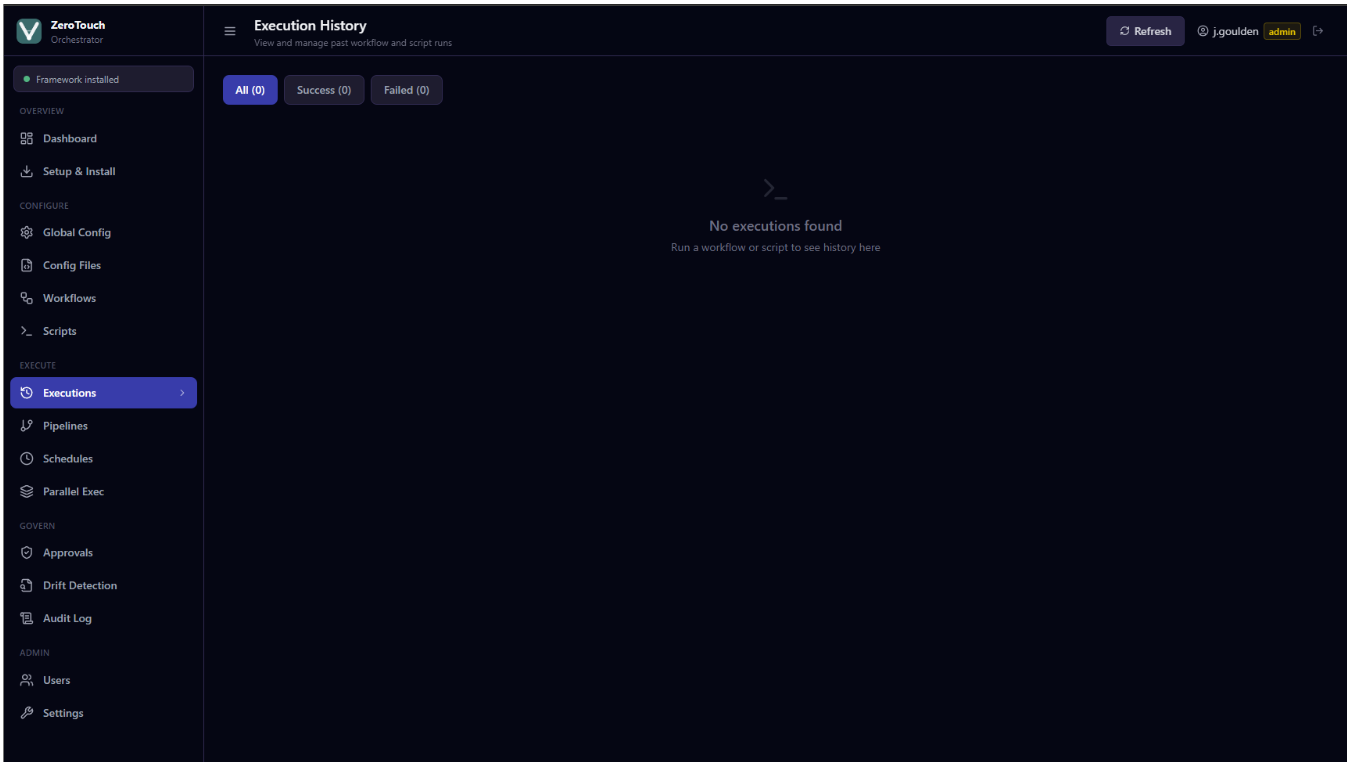

Figure: Execution History status filters and expandable rows

Filtering

Use the All / Success / Failed filter buttons to narrow the list. Each button shows the count for that status.

Re-run

Expand any execution row by clicking it. If the original config is available, a Re-run button appears alongside the config filename. Clicking Re-run opens the execution modal pre-loaded with the original YAML no form re-entry required.

> **Note:** Re-run is only available for executions recorded after v1.2.2. Earlier records do not store config content.

11. Pipelines

The Pipelines page allows workflows to be chained sequentially with automatic pass/fail gates. Each pipeline runs its steps in order if any step fails, remaining steps are skipped and the pipeline halts. This enables fully unattended end-to-end site deployments without operator intervention between steps.

Creating a Pipeline

Click New Pipeline to open the builder form.

- Enter a descriptive Pipeline Name (e.g. "Full Site Deployment Site A")
- Click Add Step to append a step to the sequence
- For each step, select the workflow from the dropdown and optionally select the config file to use
- Use the up/down arrows to reorder steps
- Click Create Pipeline to save

> **Note:** Config files listed in the step picker come from the Config Files page. Create and save your workflow configs there before building a pipeline.

Pipeline List

Saved pipelines are displayed as cards showing the pipeline name, step count, last-updated date, and a visual flow of steps connected by arrows (e.g. imaging-only → cluster-create → deploy-pc → config-pc). Each card has Run, Edit, and Delete actions.

Running a Pipeline

Click Run on any pipeline card. The pipeline execution modal opens showing:

- A step progress rail at the top each step shows its status: pending, running (spinner), success (teal), failed (red), or skipped (grey)
- A combined live terminal with step-separator headers between each workflow output
- The modal remains open until the pipeline completes or fails

Pass/fail gate behaviour:

- Every step only starts if the previous step exited with return code 0
- On first failure the pipeline halts; all remaining steps are marked Skipped
- The overall status (success / failed) is recorded in Execution History

> **Note:** Pipeline runs appear in Execution History with a [Pipeline] prefix. Webhook notifications include step-level results if a webhook URL is configured in Settings.

Typical Deployment Pipeline

Step	Workflow	Purpose
1	Imaging Only	Image nodes with AOS and AHV via Foundation Central
2	Cluster Create	Form the cluster and set DNS/NTP
3	Deploy Prism Central	Deploy PC VM to the new cluster
4	Configure Prism Central	Apply AD, SAML, NKE, security policies
5	Configure Cluster	Set containers, networks, and HA reservation

12. Schedules

The Schedules page automates workflow execution using standard 5-field cron expressions. Each schedule has a name, workflow or script target, YAML configuration, enabled/disabled state, and last-run status.

Create a schedule by selecting the workflow or script, entering the YAML config content, and choosing a cron expression. Operators and admins can create or update schedules; viewers can list schedules; only admins can delete them.

Use Run Now to trigger a schedule immediately without waiting for the next cron interval. Scheduled runs are written to Execution History and use the configured webhook URL when notifications are enabled.

13. Parallel Exec

Parallel Exec runs the same workflow across multiple sites at the same time. Each site has its own label and YAML configuration, allowing regional or multi-cluster tasks to be executed together while still preserving per-site output.

Create a run with 2 to 10 sites, then submit it from the Parallel Exec page. The page polls while the run is active and shows per-site status, return code, and expandable terminal output. Overall status is success, partial, or failed.

14. Approvals

Approvals provide an operator-to-admin gate for sensitive operations. Operators submit the requested workflow, YAML config, and notes. Admins can approve or reject the request with an optional decision note.

Pending requests appear with a sidebar count. Requests automatically expire after 24 hours if no decision is made. Creation and decision events can send webhook notifications when a webhook URL is configured.

15. Drift Detection

Drift Detection compares a saved desired ZTF config file with either the last successful applied configuration or a pasted current-state JSON/YAML snapshot from Prism Central, Foundation Central, or another source.

Results are grouped as matched, changed, missing, unexpected, or unknown. A drifted status means at least one changed, missing, or unexpected field was found. Unknown indicates no suitable last-applied baseline was available.

Admins and operators can run checks; viewers can review existing drift history. Admins can clear drift history when it is no longer needed.

16. Audit Log

The Audit Log page provides a searchable, filterable view of the structured application log. Every significant event logins, workflow executions, config saves, user creation, and settings changes is recorded with a timestamp, the acting user, and their IP address. Access is restricted to the admin role.

> **Note:** Audit Log is admin-only. Operators and viewers receive an access-denied response if they navigate to this page.

Reading the Log

Each row shows:

- Timestamp: when the event occurred (local time)
- Level badge: INFO (blue), WARNING (amber), or ERROR (red)
- Message: the event type (e.g. login_success, execution_complete, config_saved)
- User: the account that performed the action
- IP: the client IP address
- Status: success or failed where applicable

Click any row to expand it and see all additional structured fields for that event.

Filtering

- Use the search box to filter by message text, username, IP address, or action name
- Use the level buttons (ALL / INFO / WARNING / ERROR) to narrow by severity
- Click Refresh to reload the most recent entries from disk

> **Note:** The log file is located at ZTF_DATA_DIR/ztf-orchestrator.log (default: ~/.ztf-ui/ztf-orchestrator.log on Linux/macOS, C:\Users\<user>\.ztf-ui\ztf-orchestrator.log on Windows). The UI shows the most recent 500 entries by default.

Common Event Messages

Message	Triggered by
login_success	Successful user login
login_failed	Failed login attempt
execution_start	Workflow or script execution begins
execution_complete	Workflow or script finishes (includes status)
pipeline_complete	Pipeline finishes (includes step results)
execution_cancelled	Legacy/client cancellation event; v1.2.8 job cancellations are logged as job_cancel_requested
webhook_fired	Webhook notification sent successfully
webhook_failed	Webhook notification failed
schedule_run	Scheduled workflow execution starts or completes
approval_created / approval_decided	Approval request created, approved, rejected, or expired
parallel_run_complete	Parallel multi-site execution completes
drift_check	Drift detection check completes
settings_saved	Runtime, profile, repository, or notification settings changed
job_queued	Workflow or script was submitted to the durable job queue
job_cancel_requested	Admin or operator requested cancellation of a queued or running job

17. User Management

Navigate to Users in the sidebar to manage user accounts. Admin role is required to create, modify, or delete users.

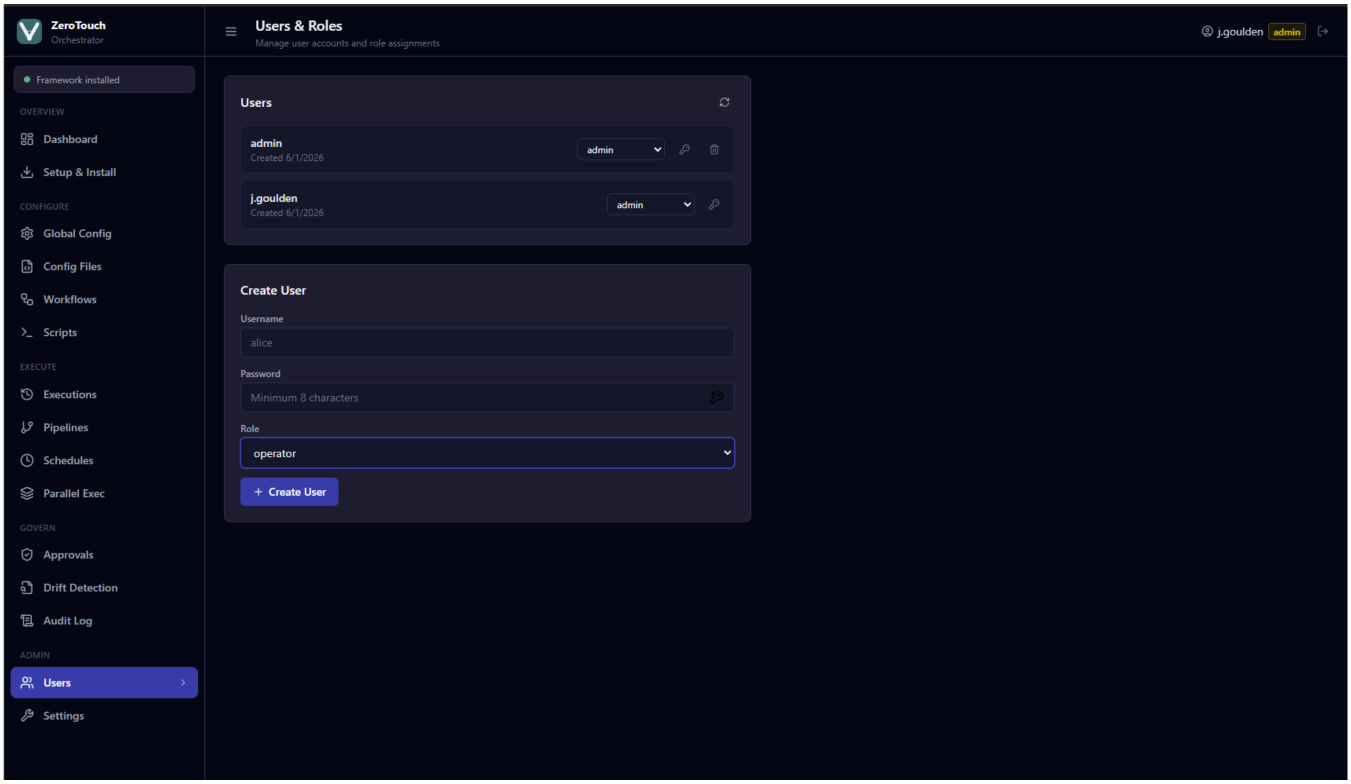

Figure: Users & Roles user list and Create User form

Roles

Role	Permissions
admin	Full access settings, global config, user management, all workflows and scripts
operator	Execute workflows and scripts, manage config files, view execution history
viewer	Read-only view configs, execution history, and system status

Creating a User

- Enter a username and password (minimum 8 characters)
- Select a role from the dropdown
- Click Create User

Changing a Role

Use the role dropdown on any existing user row. The change saves immediately.

Resetting a Password

Click the key icon on any user row to expand the password reset form. Type the new password and press Save or Enter.

> **Note:** Users cannot delete their own account. Admins can delete any other user.

18. Settings

Settings configure runtime paths, repository source, storage/backend visibility, named connection profiles, notification webhooks, and product information. Admin role is required to modify editable settings.

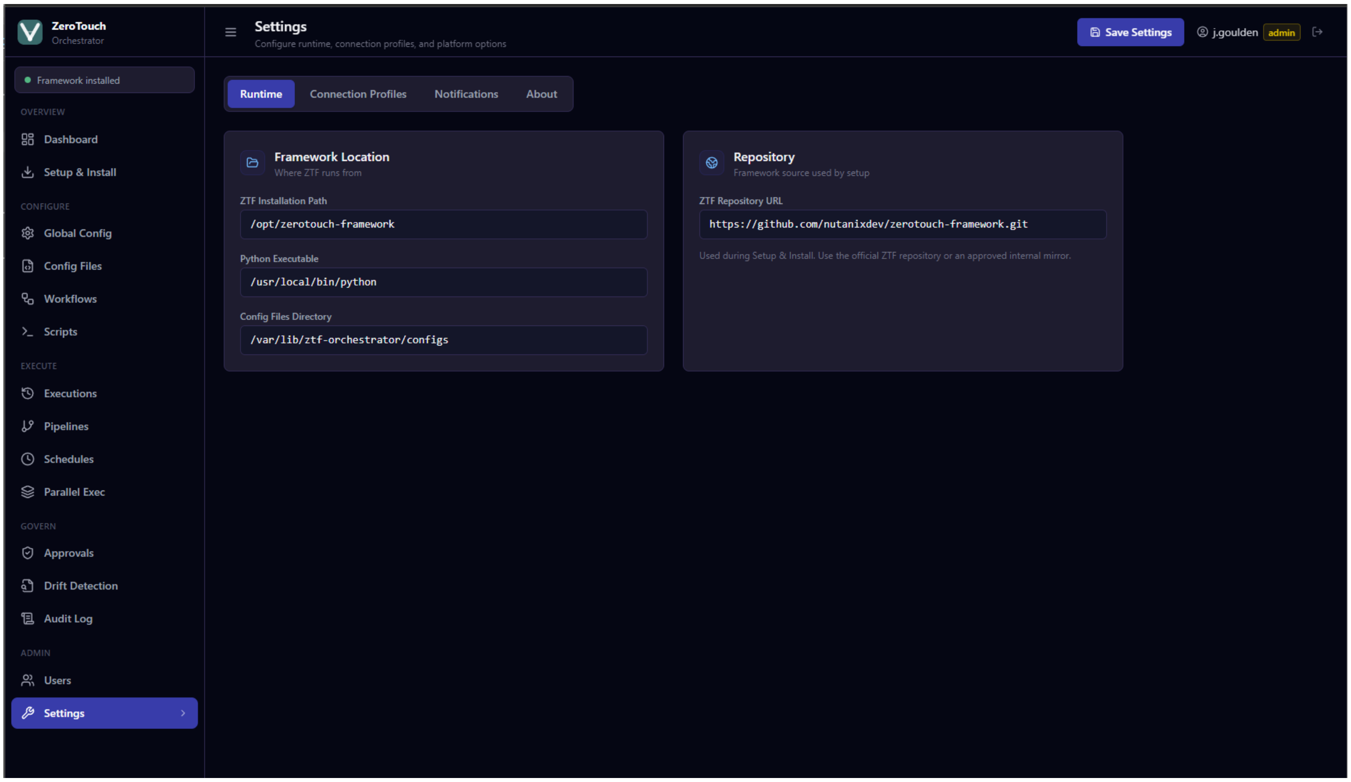

Figure: Settings runtime, storage, connection profiles, notifications, and about

Storage
The Storage tab shows the active backend, service health, data directory, database location, retention windows, ZTF installed state, and product version. Database credentials are intentionally hidden.

Framework Location

- ZTF Installation Path: full path to the cloned zerotouch-framework directory containing main.py
- Python Executable: path to the Python interpreter used to run ZTF (defaults to python3)
- Config Files Directory: where YAML/JSON config files are stored (defaults to ~/.ztf-ui/configs)

Repository

The ZTF Repository URL is used by the Setup & Install wizard. For air-gapped deployments, set this to your internal Git mirror URL.

Connection Profiles

Connection Profiles define reusable environment defaults for generated workflow YAML. A profile can represent Lab, Pre-Production, Production, a customer environment, or another operating context. Each profile can store endpoint and credential-reference defaults for Prism Central, Foundation Central, Prism Element/CVM, NCM/Calm, directory services, IPAM/Infoblox, DNS, NTP, timezone, and site code. Passwords remain in Global Config/global.yml; profiles reference credential keys rather than exposing secrets.

The active profile is used by guided workflow forms to pre-fill relevant fields. The Settings page also provides a generated ZTF defaults YAML preview so operators can verify what values will feed into workflow configuration.

Notifications (Webhook)

Enter a webhook URL to receive a POST notification when workflows, scripts, schedules, pipelines, approvals, or parallel runs complete. The payload includes workflow name, status, user, return code, and timestamp. Compatible with Slack incoming webhooks, Microsoft Teams, n8n, and any custom HTTP endpoint. Leave blank to disable.

> **Note:** Click Save Settings to apply changes. Settings take effect immediately without restarting the server.